```
    stroke: black;
    stroke-width: 1px;
}
```

We can use percentages of the screen because an SVG element can be displayed as a percentage just like div elements. But the elements drawn in the SVG won't scale to fit the SVG automatically. Instead, we'll bind functionality to `window.onresize` to redraw our visual elements (as shown in listing 9.3). We could also use the viewport attribute of an SVG element to automatically resize the graphics, but we'll want more fine-grained control of our graphics when creating data visualization applications (recall the distinction between graphical and semantic zoom discussed in chapter 7).

9.1.1 Data dashboard basics

Before we redraw anything, we need to load our data and add our charts. We'll accomplish that by calling the `startup` function on the successful completion of d3.json loading the venerable tweets.json file we worked with in chapter 5.

Listing 9.3 General dashboard function

```
function dashboard() {
  window.onresize = function(event) {
      redraw();
  };

  d3.json("tweets.json",function(error,data) {startup(data.tweets)});

  function startup(incData) {
    createSpreadsheet(incData, "#spreadsheet");

    var nestedTweets = d3.nest()
      .key(function (el) {return el.user})
      .entries(incData);

    packableTweets = {id: "root", values: nestedTweets};

    createBar(nestedTweets, "#rightSVG");
    createPack(packableTweets, "#leftSVG");
    redraw();
  };
};
```

Whenever the user adjusts the screen, we want to redraw the charts.

The spreadsheet takes the array of tweets.

The bar chart takes the nested tweets.

The circle pack takes the nested tweets in a root node.

Without calling redraw (shown in section 9.1.5), we wouldn't see any of our SVG graphics.

We'll get to `redraw()` in listing 9.8 after we first look at each individual chart, so that you can see everything that's happening on startup.

Each chart has its own function accessing the raw or processed dataset, as shown in listing 9.3. By using the same nested array for our packable tweets and our bar chart, we can determine if the data bound to SVG elements in each chart is the same. It's critical that you use the same array for both, and not use `d3.nest` twice to create an array that contains the same data but isn't the same array. That's because `d3.nest` creates new nested user elements (`"Al"`, `"Pris"`, `"Sam"`, `"Roy"`) that are treated differently